Enrollment No.: 240840116014

Practical-5

Aim: Analyze datasets using correlation, covariance, outlier detection, and distribution analysis.

Step 1: Import required libraries

```
#: Import required libraries for data analysis and visualization import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2: Create a Dataset

```
# Define the student data dictionary with relevant academic metrics
data = {
    "Student_ID": [101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115],

    "Study_Hours": [2, 3, 4, 5, 6, 7, 8, 9, 3, 4, 5, 6, 7, 10, 1],

    "Attendance": [65, 70, 75, 80, 82, 85, 90, 92, 68, 74, 78, 84, 88, 95, 40],

    "Assignment_Score": [55, 60, 65, 70, 72, 78, 82, 88, 62, 67, 71, 76, 80, 95, 35],

    "Final_Score": [50, 55, 62, 68, 72, 78, 82, 90, 58, 63, 69, 75, 81, 98, 25]
}

# Convert the dictionary into a pandas DataFrame
df = pd.DataFrame(data)

# Display the complete DataFrame
print(df)
```

```
    Student_ID  Study_Hours  Attendance  Assignment_Score  Final_Score
0          101            2          65                55           50
1          102            3          70                60           55
2          103            4          75                65           62
3          104            5          80                70           68
4          105            6          82                72           72
5          106            7          85                78           78
6          107            8          90                82           82
7          108            9          92                88           90
8          109            3          68                62           58
9          110            4          74                67           63
10         111            5          78                71           69
11         112            6          84                76           75
12         113            7          88                80           81
13         114           10          95                95           98
14         115            1          40                35           25
```

Step 3: Inspect Dataset

```
# Display the first five records of the dataset
print(df.head())
```

```
   Student_ID  Study_Hours  Attendance  Assignment_Score  Final_Score
0         101            2          65                55           50
1         102            3          70                60           55
2         103            4          75                65           62
3         104            5          80                70           68
4         105            6          82                72           72
```

```
#Display information about the dataset

print("Dataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 5 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Student_ID        15 non-null     int64
 1   Study_Hours       15 non-null     int64
```

```
 2   Attendance        15 non-null     int64
 3   Assignment_Score  15 non-null     int64
 4   Final_Score       15 non-null     int64
dtypes: int64(5)
memory usage: 732.0 bytes
```

```
#Display number of rows and columns

rows, columns = df.shape

print("Number of Rows:", rows)
print("Number of Columns:", columns)
```

```
Number of Rows: 15
Number of Columns: 5
```

```
#Generate basic descriptive statistics

print("Basic Descriptive Statistics:")
print(df.describe())
```

```
Basic Descriptive Statistics:
       Student_ID  Study_Hours  Attendance  Assignment_Score  Final_Score
count   15.000000    15.000000   15.000000         15.000000    15.000000
mean   108.000000     5.333333   77.733333         70.400000    68.400000
std      4.472136     2.581989   13.718948         14.500246    17.759504
min    101.000000     1.000000   40.000000         35.000000    25.000000
25%    104.500000     3.500000   72.000000         63.500000    60.000000
50%    108.000000     5.000000   80.000000         71.000000    69.000000
75%    111.500000     7.000000   86.500000         79.000000    79.500000
max    115.000000    10.000000   95.000000         95.000000    98.000000
```

Step 4: Calculate Correlation

```
#Calculate correlation between Study Hours and Final Score

correlation = df["Study_Hours"].corr(df["Final_Score"])

print("Correlation between Study Hours and Final Score:")
print(correlation)
```

```
Correlation between Study Hours and Final Score:
0.9626650934516625
```

Step 5: Calculate Correlation Matrix

```
#Calculate correlation among all numerical variables

correlation_matrix = df.corr(numeric_only=True)

# Display the correlation matrix
print("Correlation Matrix:")
print(correlation_matrix)
```

```
Correlation Matrix:
                  Student_ID  Study_Hours    Attendance  Assignment_Score  \
Student_ID        1.000000e+00     0.247436 -7.279626e-16          0.136585
Study_Hours       2.474358e-01     1.000000  9.201931e-01          0.961552
Attendance       -7.279626e-16     0.920193  1.000000e+00          0.981905
Assignment_Score  1.365851e-01     0.961552  9.819053e-01          1.000000
Final_Score       1.286063e-01     0.962665  9.837631e-01          0.999268

                  Final_Score
Student_ID           0.128606
Study_Hours          0.962665
Attendance           0.983763
Assignment_Score     0.999268
Final_Score          1.000000
```

Step 6: Visualize Correlation Using Scatter Plot

```
# Create a scatter plot
# to visualize the relationship between Study Hours and Final Score

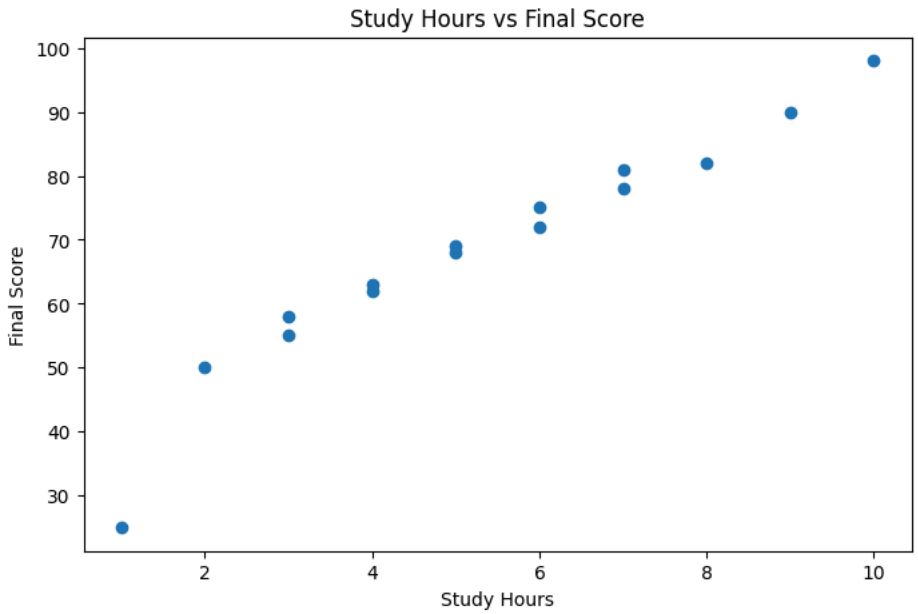
plt.figure(figsize=(8, 5))

plt.scatter(df["Study_Hours"], df["Final_Score"])

plt.xlabel("Study Hours")
plt.ylabel("Final Score")
```

```
plt.title("Study Hours vs Final Score")

plt.show()
```



Step 7: Calculate Covariance

```
# Calculate covariance between Study Hours and Final Score

covariance = df["Study_Hours"].cov(df["Final_Score"])

# Display covariance
print("Covariance between Study Hours and Final Score:")
print(covariance)
```

```
Covariance between Study Hours and Final Score:
44.14285714285714
```

Step 8: Calculate Covariance Matrix

```
# Calculate Covariance Matrix

# Select numerical columns
numerical_columns = df.select_dtypes(include="number")

# Calculate covariance matrix
covariance_matrix = numerical_columns.cov()

# Display covariance matrix
print("Covariance Matrix:")
print(covariance_matrix)
```

```
Covariance Matrix:
                  Student_ID  Study_Hours    Attendance  Assignment_Score  \
Student_ID        2.000000e+01     2.857143 -1.015061e-15          8.857143
Study_Hours       2.857143e+00     6.666667  3.259524e+01         36.000000
Attendance       -1.015061e-15    32.595238  1.882095e+02        195.328571
Assignment_Score  8.857143e+00    36.000000  1.953286e+02        210.257143
Final_Score       1.021429e+01    44.142857  2.396857e+02        257.328571

                  Final_Score
Student_ID          10.214286
Study_Hours         44.142857
Attendance         239.685714
Assignment_Score   257.328571
Final_Score        315.400000
```

Step 9: Detect Outliers Using IQR

```
# Detect Outliers Using IQR

# Select Final_Score column
final_score = df["Final_Score"]

# Calculate Q1 and Q3
```

```
Q1 = final_score.quantile(0.25)
Q3 = final_score.quantile(0.75)

# Calculate IQR
IQR = Q3 - Q1

# Calculate lower and upper bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Display the values
print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
```

```
Q1: 60.0
Q3: 79.5
IQR: 19.5
Lower Bound: 30.75
Upper Bound: 108.75
```

Step 10: Identify Outliers

```
# Select records below the lower bound
# or above the upper bound
outliers = df[
    (df["Final_Score"] < lower_bound) |
    (df["Final_Score"] > upper_bound)
]

# Display the outlier records
print("Detected Outlier Records:")
print(outliers)
```

```
Detected Outlier Records:
    Student_ID  Study_Hours  Attendance  Assignment_Score  Final_Score
14         115            1          40                35           25
```

Step 11: Count Outliers

```
# Count the total number of outliers

outlier_count = len(outliers)

print("Total Number of Outliers:")
print(outlier_count)
```

```
Total Number of Outliers:
1
```

Step 12: Visualize Outliers Using Box Plot

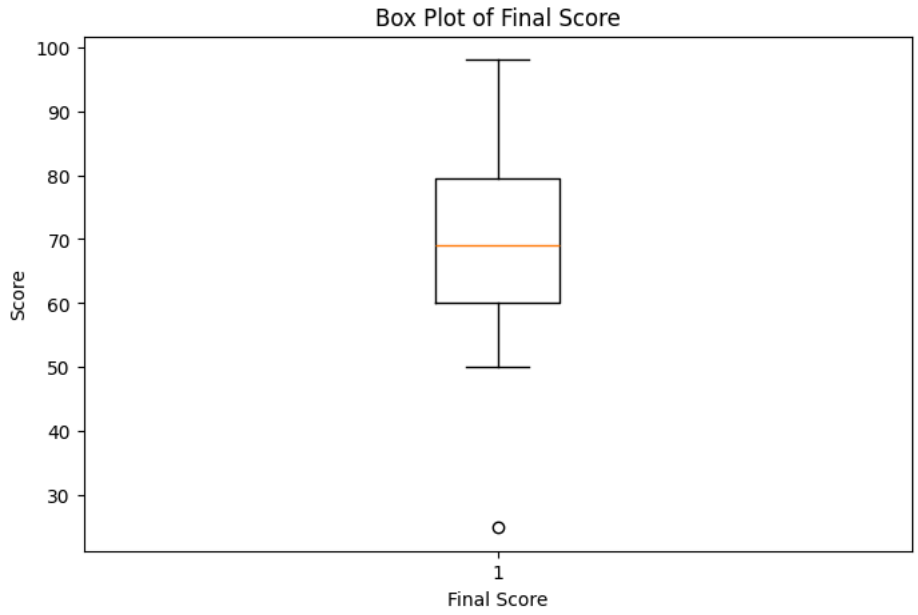
```
# Visualize Outliers Using Box Plot

plt.figure(figsize=(8, 5))

plt.boxplot(df["Final_Score"])

plt.xlabel("Final Score")
plt.ylabel("Score")
plt.title("Box Plot of Final Score")

plt.show()
```



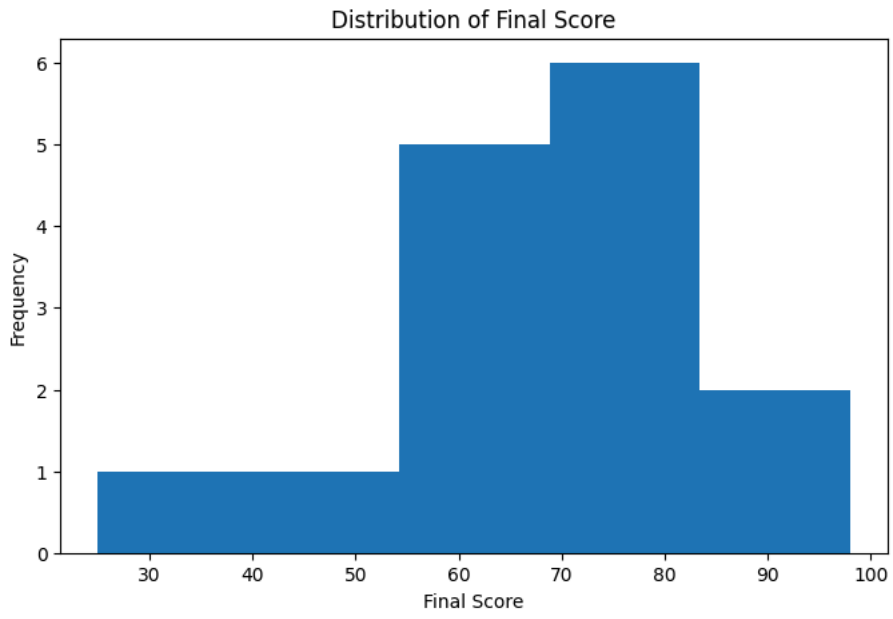
Step 13: Create a Histogram

```
# Create a histogram to observe the distribution of Final Score

# Create the histogram
plt.figure(figsize=(8, 5))
plt.hist(df["Final_Score"], bins=5)

# Add labels and title
plt.xlabel("Final Score")
plt.ylabel("Frequency")
plt.title("Distribution of Final Score")

# Display the histogram
plt.show()
```



Step 14: Analyze Distribution Using Skewness

```
# Calculate the skewness of Final Score

# Calculate skewness
skewness = df["Final_Score"].skew()

# Display the skewness value
print("Skewness of Final Score:")
print(skewness)
```

```
Skewness of Final Score:
-0.7383380071904277
```

Step 15: Calculate Kurtosis

```
# Calculate the kurtosis of Final Score

# Calculate kurtosis
kurtosis = df["Final_Score"].kurt()

# Display the kurtosis value
print("Kurtosis of Final Score:")
print(kurtosis)
```

```
Kurtosis of Final Score:
1.4936683306648835
```

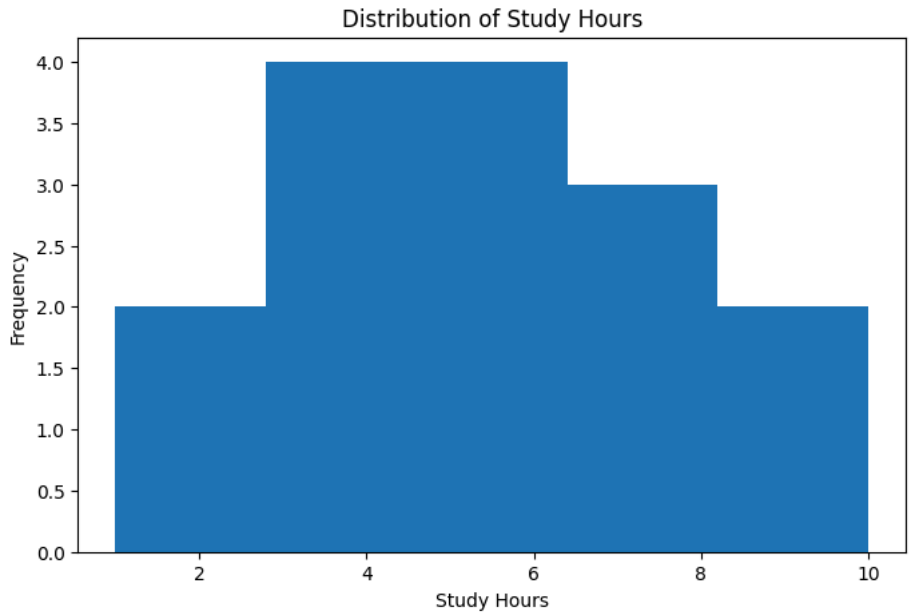
Step 16: Distribution of Study Hours

```
# Create a histogram for Study Hours

# Create the histogram
plt.figure(figsize=(8, 5))
plt.hist(df["Study_Hours"], bins=5)

# Add labels and title
plt.xlabel("Study Hours")
plt.ylabel("Frequency")
plt.title("Distribution of Study Hours")

# Display the histogram
plt.show()
```



Step 17: Compare Variables Using Box Plots

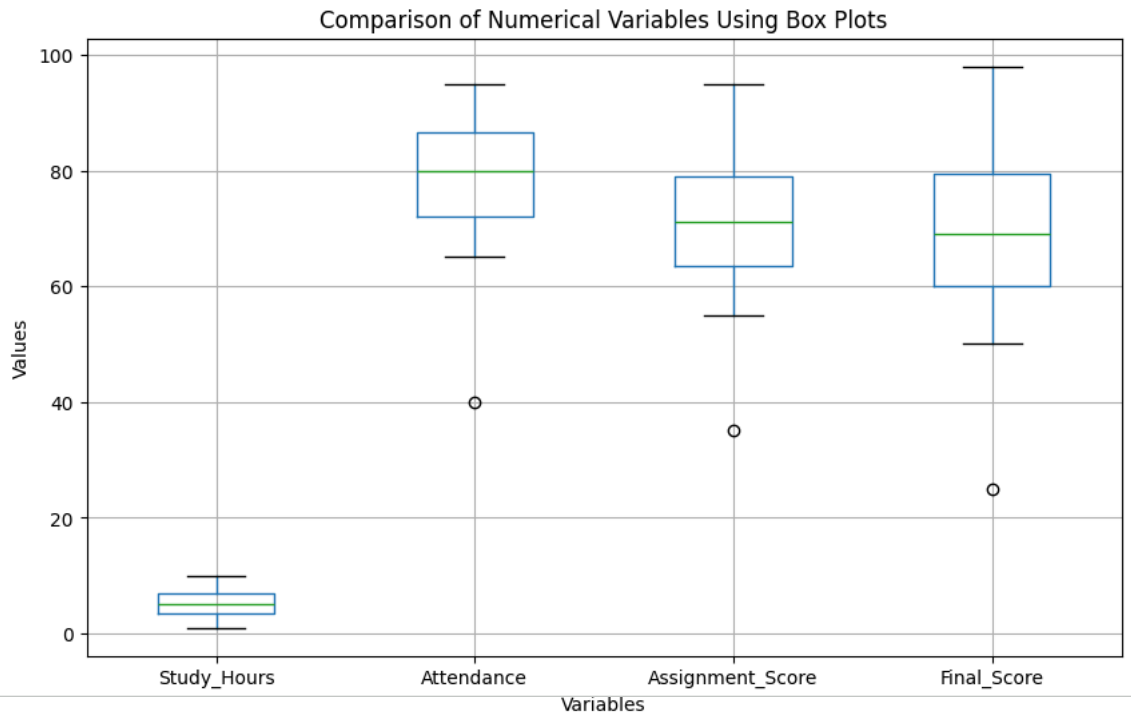
```
# Compare variables using box plots

# Select numerical variables for box plot
numerical_data = df[
    ["Study_Hours", "Attendance", "Assignment_Score", "Final_Score"]
]

# Create box plots
plt.figure(figsize=(10, 6))
numerical_data.boxplot()

# Add title and labels
plt.title("Comparison of Numerical Variables Using Box Plots")
plt.xlabel("Variables")
plt.ylabel("Values")
```

```
# Display the box plots
plt.show()
```



Step 18: Generate a Statistical Summary

```
# Generate a Statistical Summary

# Calculate the statistical summary of the dataset
statistical_summary = df.describe()

# Display the statistical summary
print("Statistical Summary:")
print(statistical_summary)
```

```
Statistical Summary:
       Student_ID  Study_Hours  Attendance  Assignment_Score  Final_Score
count   15.000000    15.000000   15.000000         15.000000    15.000000
mean   108.000000     5.333333   77.733333         70.400000    68.400000
std      4.472136     2.581989   13.718948         14.500246    17.759504
min    101.000000     1.000000   40.000000         35.000000    25.000000
25%    104.500000     3.500000   72.000000         63.500000    60.000000
50%    108.000000     5.000000   80.000000         71.000000    69.000000
75%    111.500000     7.000000   86.500000         79.000000    79.500000
max    115.000000    10.000000   95.000000         95.000000    98.000000
```